

Module 03: Perception — Implementing State Inference

Course: Active Inference 101 | **Unit:** Implementation | **Audience:** First-semester undergraduates

Learning Objectives

1. Implement **variational message passing** for state inference in a hierarchical model.
2. Code **precision-weighted prediction errors** and analyze their behavior.
3. Visualize belief updating in real time.

Introduction

Module 02 gave us a basic Bayesian update. Real Active Inference uses iterative variational inference — updating beliefs through message passing until convergence. This module implements the full perceptual inference algorithm.

Key Concepts

1. Variational Message Passing

Instead of a single Bayesian update, real inference iterates:

```

def variational_inference(A, B, D, observations, actions, num_iter=16):
    """
    Run variational message passing to infer hidden states.

    Parameters:
        A: observation model (num_obs x num_states)
        B: transition model (num_states x num_states x num_actions)
        D: initial state prior (num_states,)
        observations: list of observation indices
        actions: list of action indices
        num_iter: number of iterations per time step

    Returns:
        qs: list of posterior beliefs at each time step
    """
    T = len(observations)
    num_states = A.shape[1]

    # Initialize beliefs
    qs = [np.ones(num_states) / num_states for _ in range(T)]

    for iteration in range(num_iter):
        for t in range(T):
            # Message from observations (likelihood)
            ln_A = np.log(A[observations[t], :] + 1e-16)

            # Message from past (prior or transition)
            if t == 0:
                ln_prior = np.log(D + 1e-16)
            else:
                ln_prior = np.log(B[:, :, actions[t-1]] @ qs[t-1] + 1e-16)

```

```
# Message from future (if available)
if t < T - 1:
    ln_future = np.log(B[:, :, actions[t]].T @ qs[t+1] + 1e-1)
else:
    ln_future = np.zeros(num_states)

# Combine messages and normalize (softmax)
ln_qs = ln_A + ln_prior + ln_future
qs[t] = np.exp(ln_qs - ln_qs.max())
qs[t] = qs[t] / qs[t].sum()

return qs
```

2. Prediction Error Computation

```
def compute_prediction_errors(A, B, qs, observations, actions):  
    """  
    Compute prediction errors at each time step.  
  
    Returns:  
        state_pe: prediction errors for state transitions  
        obs_pe: prediction errors for observations  
    """  
    T = len(observations)  
    state_pe = []  
    obs_pe = []  
  
    for t in range(T):  
        # Observation prediction error  
        predicted_obs = A @ qs[t]  
        actual_obs = np.zeros(A.shape[0])  
        actual_obs[observations[t]] = 1.0  
        obs_pe.append(actual_obs - predicted_obs)  
  
        # State prediction error (if t > 0)  
        if t > 0:  
            predicted_state = B[:, :, actions[t-1]] @ qs[t-1]  
            state_pe.append(qs[t] - predicted_state)  
        else:  
            state_pe.append(np.zeros(qs[t].shape))  
  
    return state_pe, obs_pe
```

3. Precision-Weighted Errors

```
def precision_weighted_update(prediction_error, precision):  
    """  
    Weight prediction error by precision.  
  
    Parameters:  
        prediction_error: raw prediction error vector  
        precision: scalar precision (inverse variance)  
  
    Returns:  
        weighted_error: precision-weighted prediction error  
    """  
    return precision * prediction_error
```

```
# Example: High precision (reliable sense) vs. low precision (unreliable)  
error = np.array([0.3, -0.2])  
high_prec = precision_weighted_update(error, precision=10.0)  
low_prec = precision_weighted_update(error, precision=0.1)  
print(f"High precision error: {high_prec}")    # [3.0, -2.0]  
print(f"Low precision error: {low_prec}")      # [0.03, -0.02]
```

4. Free Energy Computation

```
def compute_free_energy(A, qs, observation, prior):  
    """  
    Compute variational free energy at one time step.  
  
     $F = D_{KL}(q(s) || \text{prior}(s)) - E_q[\log P(o|s)]$   
     $F = \text{Complexity} - \text{Accuracy}$   
    """  
    # Accuracy:  $E_q[\log P(o|s)]$   
    accuracy = (qs * np.log(A[observation, :] + 1e-16)).sum()  
  
    # Complexity:  $D_{KL}(q || \text{prior})$   
    complexity = (qs * (np.log(qs + 1e-16) - np.log(prior + 1e-16))).sum()  
  
    F = complexity - accuracy  
    return F, accuracy, complexity
```

Summary

Full perceptual inference uses variational message passing — iteratively combining messages from past, present observations, and future expectations. Prediction errors are computed as mismatches between expected and actual observations/states. Precision weighting controls the influence of each error. Free energy quantifies the quality of inference and decomposes into accuracy and complexity.

Further Reading

- Parr, T. & Friston, K. J. (2019). Generalised free energy and active inference. *Biological Cybernetics*, 113(5), 495-513.
- Bogacz, R. (2017). A tutorial on the free-energy framework. *Journal of Mathematical Psychology*, 76, 198-211.